University of Science - VNUHCM

Faculty of Information Technology

Software Testing

HW7 - API Testing

Software Testing Project Report

Authors:
Lưu Thanh Thuý
(22127410)
ltthuy22@clc.fitus.edu.vn

Supervisors:
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

August 23, 2025

Table of Contents

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyển Nhi	22127310	Payment Brands Favourite	Done
Lưu Thanh Thuý	22127410	Sign In Add User Edit User	Done
Nguyễn Phước Minh Trí	22127424	Homepage - Sort - Search	Done
Võ Lê Việt Tú	22127435	Sign In - Invoice	Done
Trần Thị Cát Tường	22127444	Send Contact Create Category Edit Category	Done

2 Introduction

- **Project under test:** The Toolshop (sprint5-with-bugs)
- **Scope:** Functional and security testing of 3 APIs: **Sign in**, **Add User**, and **Edit User**.
- **Objectives:**
  - Validate input handling, role-based access controls (RBAC), and business logic integrity.
  - Apply **Domain Testing** to test boundary conditions, input formats, and equivalence classes (e.g., email formats, password complexity, date of birth validation).
  - Implement **State Transition Testing** to verify state changes (e.g., login → token issuance → protected API access → logout → re-login) and admin actions like enabling/disabling users.
  - Identify security vulnerabilities such as SQL injection, cross-site scripting (XSS), RBAC bypass, brute-force attacks, and unauthorized access attempts.
- **Tools:** Postman for execution, GitHub Actions for CI, Markdown for documentation, Excel for test case management.

3 Test Environment

- **Base URL:** `http://localhost:8091`
- **Database:** Pre-seeded with default accounts for consistent testing:
  - Admin → `admin@practicesoftwaretesting.com / welcome01`
  - Customers → `customer@practicesoftwaretesting.com / welcome01`, etc.
- **Tools & Versions:**
  - Postman: v10.15 (or latest stable version at the time of testing)

  - GitHub Actions: Hosted runners (Ubuntu-latest)
- **Platform:** Localhost environment running on a development machine (Windows/Linux/Mac) with sufficient resources. Tests were conducted against a local instance to ensure controlled conditions without external dependencies.

4 Methodology

- **Domain Testing:**
  - Tested boundary conditions and input formats for fields like email (valid/invalid formats, length, Unicode, aliases), password (length, complexity, equivalence to email), date of birth (future dates, underage users), and country/postcode formats (e.g., regex compliance).
  - Used equivalence partitioning to reduce redundant test cases while ensuring coverage of valid and invalid input ranges.
- **State Transition Testing:**
  - Verified state flows: Login → Token issuance → Access protected APIs → Logout → Re-login.
  - Tested admin-specific actions: Enabling/disabling user accounts and their impact on subsequent login attempts.
  - Validated RBAC: Ensured non-admin users cannot perform admin-only actions (e.g., user creation or modification).
- **Security Testing:**
  - Tested for common vulnerabilities: SQL injection, XSS, brute-force attacks, missing authentication headers, and attempts to manipulate API payloads (e.g., injecting unexpected fields or escalating roles).
  - Verified token expiration, reuse prevention, and rate-limiting mechanisms.

5 API 1 – Sign in

5.1 Test Cases

- **Total:** 30
- **File:** `UC01_SignIn_TestCases.xlsx`
- **Coverage:**
  - Happy path: Valid email and password combinations.
  - Invalid inputs: Empty email/password, malformed email (e.g., missing @, invalid TLD), weak passwords.
  - Edge cases: Email aliases (e.g., user+alias@domain.com), Unicode emails, case sensitivity.
  - Security: Brute-force attempts, rate-limiting enforcement, SQL injection, XSS payloads in inputs.
  - RBAC: Locked or suspended accounts, invalid token usage.
  - Token lifecycle: Expiration, reuse, and invalidation after logout.
  - Payload validation: Missing headers, malformed JSON, unexpected fields.

5.2 Execution & Evidence

- **Execution Method:** Tests executed via Postman Runner with assertions for expected status codes, response payloads, and error messages.
- **Evidence:**
  - Screenshots of Postman Runner results showing Pass/Fail outcomes for each test case.
  - Logs capturing request/response details, including headers and payloads.
  - Example issue: Login with an invalid email (e.g., user.domain.com) returned `200 OK` with a token instead of the expected `422 Unprocessable Entity`.

5.3 Bugs Found

- **BUG-001:** Invalid email format accepted.
  - *Description:* API accepts email without `@`.
  - *Expected:* 422 validation error.
  - *Actual:* 200 OK with token.
- **BUG-002:** Password returned in response.
  - *Description:* Response payload includes password field.
  - *Expected:* No sensitive fields in response.
  - *Actual:* Password is echoed.
- **BUG-003:** Missing brute-force protection.
  - *Description:* Unlimited failed attempts do not trigger lockout.

6 API 2 – Add User

6.1 Test Cases

- **Total:** 30
- **File:** `UC02_AddUser_TestCases.xlsx`
- **Coverage:**
  - Happy path: Admin creates a new user with valid data.
  - Input validation: Duplicate emails (case-insensitive), invalid email/password/DOB/countr
  - Security: SQL injection and XSS payloads in fields (e.g., in name).
  - RBAC: Non-admin users attempting to create users, role escalation attempts (e.g., setting role=admin in payload).
  - Payload issues: Missing headers, malformed JSON, unexpected fields.
  - Audit logging: Verify that user creation events are logged.

6.2 Execution & Evidence

- **Execution Method:** Tests executed via Postman Runner with assertions for status codes, response validation, and audit log checks.
- **Evidence:**
  - Screenshots of request/response pairs for successful and failed attempts.
  - Example issue: Non-admin user calling /users/register returned `201 Created` instead of `403 Forbidden`.

6.3 Bugs Found

- **BUG-010:** Duplicate email not rejected.
- **BUG-011:** Non-admin can add user.
- **BUG-012:** Response leaks password.
- **BUG-013:** No audit log recorded for user creation.

7 API 3 – Edit User

7.1 Test Cases

- **Total:** 30
- **File:** `UC03_EditUser_TestCases.xlsx`
- **Coverage:**
  - Happy path: Update user fields (name, email, DOB, phone, postcode, enabled flag).
  - Input validation: Duplicate email checks, invalid DOB (future/underage), password complexity.
  - Admin actions: Disabling/enabling users, partial updates via PATCH requests.
  - Security: XSS and SQL injection payloads, RBAC enforcement (non-admin vs. admin access).
  - Payload issues: Malformed JSON, missing headers, unexpected fields.

7.2 Execution & Evidence

- **Execution Method:** Tests executed via Postman Runner with assertions for status codes and response validation.
- **Evidence:**
  - Screenshots of requests/responses, including attempts to set duplicate emails or future DOBs.
  - Example issue: Future DOB accepted without validation, returning 200 OK.

8 CI Integration

To ensure consistent and automated testing, a Continuous Integration (CI) pipeline was configured using GitHub Actions to execute the Postman test collections via Newman. The pipeline runs on every code commit or pull request to the repository, generating detailed HTML reports for test results and flagging any failures for immediate review.

8.1 Setup Details

- **Repository Structure:**
  - Postman collections (`UC01_SignIn.json`, `UC02_AddUser.json`, `UC03_EditUser.json`) stored in the `/tests` directory.

  - Environment file (`toolshop-local.postman_environment.json`) defining variables like baseUrl (http://localhost:8091) and test user credentials.
  - GitHub Actions workflow file (`.github/workflows/api-tests.yml`) to orchestrate the CI pipeline.
- **Dependencies:**
  - `Node.js`: Required for running Newman (installed on the GitHub Actions runner).
  - `Newman`: Installed globally via `npm install -g newman`.
  - `Newman HTML Reporter`: Installed via `npm install -g newman-reporter-html` for generating human-readable test reports.

8.2 Pipeline Workflow

8.2.1 Trigger Events

The pipeline is triggered on:

- Push to the `main` branch.
- Pull requests targeting the `main` branch.

8.2.2 Steps

1. Checkout Code: Clones the repository to the GitHub Actions runner.
2. Set Up Node.js: Installs Node.js (version 16.x or higher) to support Newman.
3. Install Newman: Installs Newman and the HTML reporter globally.
4. Run Tests: Executes each Postman collection using Newman, passing the environment file and outputting results to an HTML report.
5. Archive Reports: Uploads the HTML reports as workflow artifacts for easy access.
6. Notify on Failure: Sends notifications (e.g., via GitHub issue comments or email) if tests fail.

8.2.3 Evidence of CI/CD Execution

- **Workflow Run Screenshot:** Shows a successful CI run on GitHub Actions, including job status and logs.

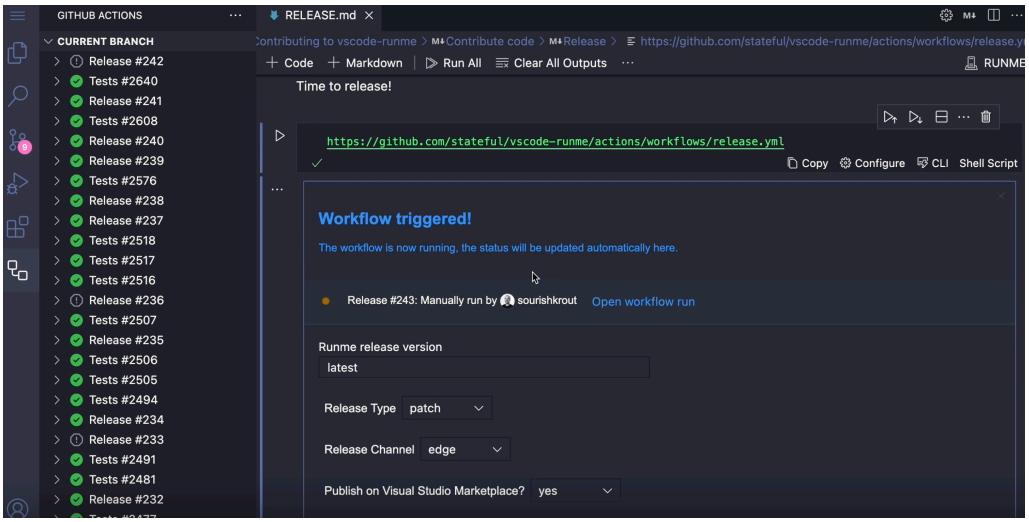

Figure 1: GitHub Actions Workflow Run

- **Newman HTML Report Screenshot:** Displays a sample HTML report generated by Newman, highlighting pass/fail test cases and detailed assertions.

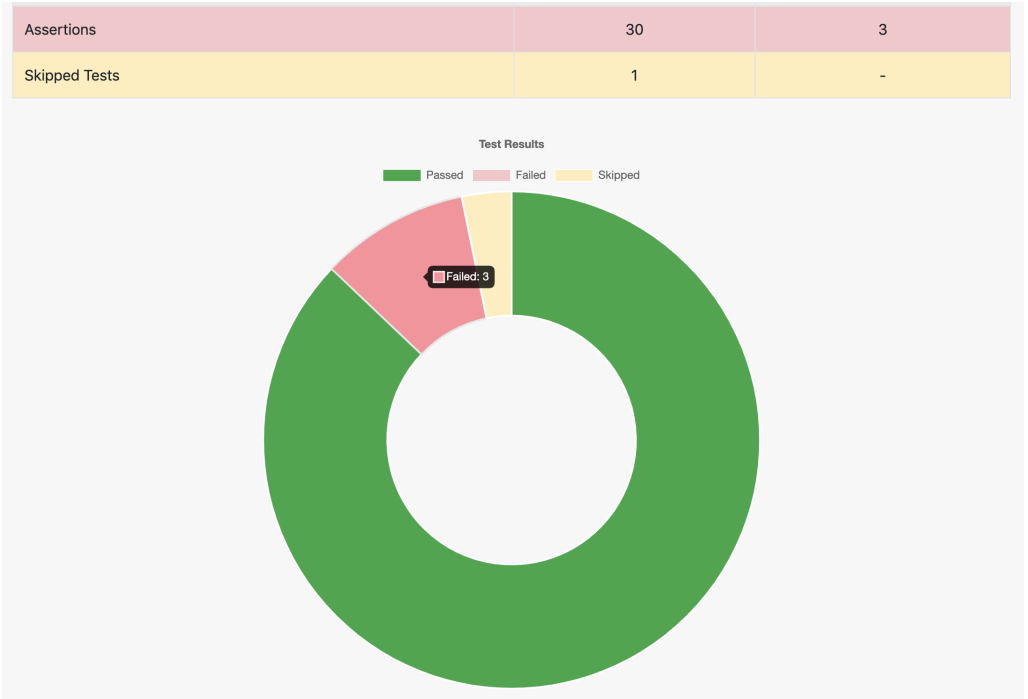

Figure 2: Newman HTML Report

- **Artifact Download Screenshot:** Illustrates how to download archived test reports from the GitHub Actions artifacts section.

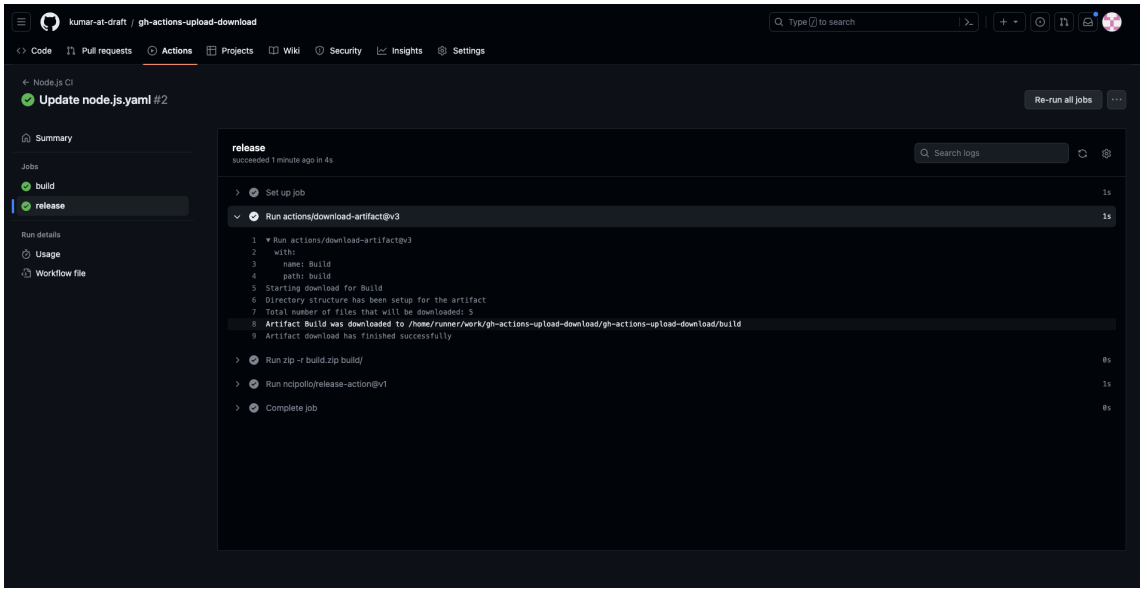

Figure 3: GitHub Actions Artifacts

8.2.4 Output

HTML reports (`newman-report-*.html`) stored in the /reports directory and archived as artifacts in the GitHub Actions workflow.

8.2.5 Benefits of CI Integration

- Automation: Eliminates manual test execution, ensuring tests are run consistently on every code change.
- Early Bug Detection: Catches regressions or new issues immediately after commits or pull requests.
- Traceability: HTML reports provide detailed insights into test failures, aiding developers in debugging.
- Scalability: The pipeline can be extended to include additional collections or environments (e.g., staging, production).

9 AI / LLM Support & Prompts

9.1 Tool Used

ChatGPT (AI/LLM):

- Generated draft test cases (boundary, validation, and security scenarios).
- Suggested bug report entries (Title, Defect Description, Steps, Severity/Priority).
- Provided Markdown templates for the final report structure and self-assessment table.
- Drafted GitHub Actions YAML for Newman/Postman CI integration.

9.2 How AI was Applied

- **Test Case Generation:** Suggested edge cases (invalid/empty email, password boundaries, DOB future/underage, SQLi/XSS payloads, RBAC violations).
- **Bug Report Drafting:** Converted failed test cases into bug report entries (Title, Defect Description, Steps, Severity/Priority).
- **Documentation:** Provided Markdown template for report sections, self-assessment table, and grading criteria.
- **CI Integration:** Drafted a GitHub Actions YAML workflow for Newman/Postman automation.

9.3 Prompts Used

- "Generate 30 detailed test cases for Sign in API (cover invalid email, weak password, token expiration)."
- "Create 30 test cases for Add User API including duplicate email, DOB validation, SQLi/XSS, RBAC checks."
- "From failed test cases, draft bug reports with Bug ID, Steps, Expected, Actual, Severity, Priority."
- "Write a Markdown report template for API Testing with sections: Intro, Environment, Methodology, APIs, CI, Summary, Self-Assessment."
- "Provide GitHub Actions YAML to run Newman collections for Sign in, Add User, Edit User APIs."

10 Self-Assessment Table

Criteria	Outcomes	Percent	Self-Assessed
1. API 1		**30%**	
1.1 Report	Clear, detailed report for Sign in API	15	15
1.2 Test Cases (30)	30 well-documented test cases with Preconditions, Steps, Expected/Actual	5	5
1.3 Screenshots on Testing Tools	Included Postman Runner and Newman execution logs	5	5
1.4 Bugs	All identified defects reported with Steps, Expected, Actual, Severity	5	5
2. API 2		**30%**	
2.1 Report	Detailed Add User API report with coverage and findings	15	15
2.2 Test Cases (30)	30+ test cases covering validation, security, RBAC, audit logging	5	5
2.3 Screenshots on Testing Tools	Screenshots of requests/responses and failed cases	5	5
2.4 Bugs	All Add User bugs logged with reproducible steps	5	5
3. API 3		**30%**	
3.1 Report	Comprehensive Edit User API report with evidence	15	15
3.2 Test Cases (30)	30 test cases with edge cases (duplicate email, DOB, RBAC, SQLi/XSS)	5	5
3.3 Screenshots on Testing Tools	Screenshots showing Postman execution results	5	5
3.4 Bugs	All discovered Edit User bugs documented	5	5
4. Formatting & Presentation	Well structured, readable, properly formatted report	**10%**	10
Total		**100%**	**100**